

GitLab application-side reindexing

GitLab comes with an application-side cronjob to execute database reindexing automatically and in background. This runbook links a couple relevant resources and documentation for this and contains steps to cancel reindexing actions during an emergency.

What is reindexing?

Please refer to `pg_repack.md#what-is-bloat` for more background on bloat. Application-side reindexing is a background process that automatically deals with standard btree postgres indexes. It currently only supports “non-special” indexes, i.e. it does not support primary key indexes, unique indexes, expression indexes etc.. However, the majority of indexes is supported.

From a workflow perspective, when the reindexing kicks off, it estimates index bloat for all supported indexes using a heuristic. It then chooses the top N indexes by their bloat level and starts to recreate those. The recreating phase is similar to what `pg_repack` does under the hoods:

1. It creates a new temporary index on the sides and
 2. then performs a renaming operation to swap indexes.
- (1) is low-risk in terms of locks but incurs additional disk IO to create the replacement index. (2) needs a elevated lock on the table to perform the renaming operations. We have guards in place, so this does not end up stalling the site for a longer period of time. If this guard doesn't work out, the process is cancelled and cleaned up automatically (so we are not forcing the lock). It will be retried upon the next invocation.

Schedule

The schedule is configured through Chef on the deploy VMs. The cronjob configuration is managed by Omnibus GitLab via the `database_reindexing` setting in `gitlab.rb`.

Configuration location:

- The cronjob is defined in `/var/opt/gitlab/crond/database-reindexing` on deploy VMs
- Configuration is set via `gitlab_rails['database_reindexing']` in `gitlab.rb`
- The Omnibus recipe that enables this is `gitlab::database_reindexing_enable`

Schedule:

- Runs on deploy VMs in `gprd` and `gstg` environments
- Schedule: `12 * * * 0,6` (every hour at 12 minutes past, on Saturdays and Sundays) - definition

- Executes: `/opt/gitlab/bin/gitlab-rake gitlab:db:reindex`

Important notes:

- The deploy VMs connect directly to the primary database node (not through PgBouncer)
- The `gitlab:db:reindex` task performs both automatic reindexing and async index creation operations
 - This Rake task has limitations on the job’s workload that are defined inside the GitLab codebase
 - It will create at most 2 async indexes and re-index at most 2 existing indexes
 - The Rake task identifies what to re-index using an index bloat heuristic
- Logs are available in `/var/log/gitlab/crond/` on the deploy VMs and are ingested into Kibana
 - Reindexing logs: <https://log.gprd.gitlab.net/app/r/s/3rNMD>
 - Async index creation logs: <https://log.gprd.gitlab.net/app/r/s/g460p>

Future migration:

- There is an ongoing effort to migrate these operations from deploy VMs to Kubernetes
- See [delivery#21650](#) for details on the migration plan

How automation picks indexes

In order to pick relevant indexes for reindexing, the job uses a bloat heuristic to determine bloat levels for all indexes. Any run of the job picks the 2 most bloated indexes by their relative bloat level.

The bloat heuristic is only available for btree and GiST indexes, GIN indexes are not supported.

Overall, the following limitations apply currently:

1. Type: Only btree and GiST indexes
2. Maximum size: Automatic reindexing only for indexes < 100 GB current size.

Explicit reindexing queue

In order to schedule specific reindexes for a rebuild, we can enqueue those indexes in the “reindexing queue”. This queue is consumed and emptied before any indexes are being picked from the bloat heuristic described above. This can be handy for indexes that are not supported, e.g. GIN indexes or indexes beyond a certain size (see above).

As an example, let's schedule `public.index_merge_request_diff_commits_on_sha` for a rebuild:

```
$ gitlab-rake gitlab:db:enqueue_reindexing_action[public.index_merge_request_diff_commits_on_sha]
```

```
Queued reindexing action: queued action [ id = 1, index: public.index_merge_request_diff_commits_on_sha ]
There are 1 queued actions in total.
```

After having enqueued the index for a rebuild, the subsequent run of `gitlab-rake gitlab:db:reindex` is going to execute the rebuild for this and other queued indexes, if any (up to 2 indexes in total per execution). If the queue is empty, we fall back to picking indexes automatically.

The queue can be examined, for example:

```
gitlabhq_production=# select * from postgres_reindex_queued_actions WHERE state = 0;
 id | index_identifier | state | created_at
-----+-----+-----+-----
  1 | public.index_merge_request_diff_commits_on_sha | 0 | 2021-11-04 10:00:00.000000
```

States for these queued actions include:

```
state = 0: queued
state = 1: done
state = 2: failed
```

How to monitor

Grafana gets annotations for any reindexing action that is happening. As an example, the PostgreSQL Overview has those enabled for production. Annotations have tags `reindex` and `gprd` or `gstg` among others, so this can be filtered upon.

In order to understand ongoing and historic reindexing actions, one can also peek at the tracking table in PostgreSQL (using a psql session):

```
gitlabhq_production=# select * from postgres_reindex_actions order by action_start desc limit 10;
 id | action_start | action_end | ondisk_size_bytes_stored
-----+-----+-----+-----
1252 | 2021-01-29 13:00:18.806929+00 | 2021-01-29 13:00:26.122366+00 | 156024
1251 | 2021-01-29 12:57:15.52368+00 | 2021-01-29 13:00:18.666019+00 | 806748
1250 | 2021-01-29 12:20:39.034396+00 | 2021-01-29 12:20:49.181661+00 | 163807
1249 | 2021-01-29 12:14:32.051548+00 | 2021-01-29 12:20:38.900695+00 | 2526797
1248 | 2021-01-29 11:18:23.072552+00 | 2021-01-29 11:18:38.730528+00 | 123117
1247 | 2021-01-29 11:14:24.531267+00 | 2021-01-29 11:18:22.915425+00 | 2256084
1246 | 2021-01-29 10:14:56.482897+00 | 2021-01-29 10:15:25.278484+00 | 180912
1245 | 2021-01-29 10:14:13.057911+00 | 2021-01-29 10:14:56.421322+00 | 537168
1244 | 2021-01-29 09:14:31.813057+00 | 2021-01-29 09:14:37.906965+00 | 76464
1243 | 2021-01-29 09:14:25.056542+00 | 2021-01-29 09:14:31.749049+00 | 76832
(10 rows)
```

The **state** column translates as follows:

1. 0 - **started**
2. 1 - **finished**
3. 2 - **failed**

How to stop

In order to disable the cronjob, we can disable the **database_reindexing** feature flag (set to false). Further invocations of the cronjob will become a no-op until it is enabled again.

However, this does *not* affect an ongoing reindexing operation. If this operation needs to be stopped in an emergency situation, the following steps can be considered:

Cancel the postgres backend performing the index rebuild

1. Log into the postgres primary using psql
2. Check current activity: `select pid, query from pg_stat_activity where query ~* 'create index';`
3. In order to cancel the index build, signal the PG backend: `select pg_cancel_backend(pid) from pg_stat_activity where query ~* 'create index' and pid <> pg_backend_pid()`
4. As a last resort, terminate the PG backend (this translates to SIGKILL and should be used only as a last resort and upon review): `select pg_terminate_backend(pid) from pg_stat_activity where query ~* 'create index' and pid <> pg_backend_pid()`

Cancel the process on the deploy node

Log into the deploy host and terminate the reindexing process: `pgrep -f gitlab:db:reindex | xargs kill`

Documentation

1. Omnibus docs
2. GitLab docs (draft)

Metrics

1. Long-term tamland forecast (internal link)